

INFO145 : Diseño y Análisis de Algoritmos

Proyecto Semestral: Técnicas de Representación y Compresión en Arreglos Ordenados

Profesor: Héctor Ferrada Escobar
Auxiliares: Carolina Obreque & Benjamín Parra

Primer semestre 2026

1. Contexto y Objetivos

El presente proyecto semestral tiene como propósito evaluar empírica y teóricamente el espacio de almacenamiento y el tiempo de ejecución en estructuras de datos. Los equipos de trabajo deberán implementar, medir y comparar diferentes estrategias para representar un arreglo ordenado de gran magnitud. El diseño empezará con una representación explícita estándar, luego una codificación llamada Gap-Coding apoyada por índices de muestreo, hasta llegar a técnicas de compresión asignadas específicamente a cada grupo.

Objetivos Específicos:

- Implementar estructuras de datos que minimicen el uso de espacio.
- Diseñar algoritmos de búsqueda adaptados a representaciones comprimidas.
- Cuantificar teórica y empíricamente la complejidad espacial y temporal. (Se debe medir el tiempo al realizar varias búsquedas no basta con solo una)

2. Especificaciones

2.1. Caso 1: Representación Explícita (Línea Base)

Consiste en dos arreglos de gran tamaño ordenados de menor a mayor, deben ser inicializados con valores aleatorios ordenados, siguiendo una distribución:

1. Lineal (Puede ser definiendo $A[0] = \text{rand}()$, $A[i] = A[i-1] + \text{rand()} \% \epsilon$)
2. Normal (Puede utilizar librerías si así lo requiere) con distintas propuestas de desviaciones estándar.

Una vez inicializadas las estructuras, se deben realizar búsquedas binarias y medir su tiempo tanto teóricamente como experimentalmente para una gran cantidad de datos. Los arreglos servirán como base de control para comparar el tiempo de respuesta y el espacio utilizado de los Casos 2 y 3.

2.2. Caso 2: Representación con Gap-Coding

Se representan ambos arreglos con Gap-Coding, los arreglos originales se vuelven inaccesible, en su lugar, se utilizara una estructura adicional llamada “sample” para ayudar a la búsqueda. Para representar un arreglo utilizando Gap-Coding se debe calcular la diferencia entre cada elemento consecutivo del arreglo **ordenado**. El primer elemento del arreglo resultante será el mismo que el primer elemento del arreglo original.

Ejemplo, para un arreglo $X = [2, 7, 10, 12, 12, 16]$ los gaps serian $GC(X) = [2, 5, 3, 2, 0, 4]$

- **Representación:** Para un arreglo ordenado A , se define GC , donde $GC[0] = A[0]$ y $GC[i] = A[i] - A[i - 1]$ para $i > 0$. Esta estructura adicional se utiliza para reconstruir al arreglo original sin acceder a este.
- **Índice de Muestreo (*Sample*):** Estructura auxiliar para permitir la búsqueda binaria (ya que en GC no hay acceso aleatorio en $\mathcal{O}(1)$ a los valores originales).
 - Defina un tamaño de muestra m ($m \ll n$) y un salto b (estático o paramétrico). Puede proponer b como variable, por ejemplo, $m = \log_3 n$ con $b = 3^j$.
 - **Ejemplo:** Si $A = [2, 7, 10, 12, 12, 16]$ con $m = 3$ y $b = 2$, con $b = \Delta i = \frac{n}{m}$, el sample sería $[2, 10, 12]$.
 - **Operación:** Búsqueda binaria sobre el *Sample* para hallar un rango $[L, R]$, seguida de una decodificación secuencial sobre GC en dicho intervalo para encontrar la posición del elemento.

2.3. Caso 3: Compresión de arreglo de gaps y Trade-off Espacio-Tiempo

En esta última etapa, se deben comprimir los arreglos que ya estaban representados con Gap-Coding en el Caso 2, utilizando una técnica de compresión entregada mas adelante.

El objetivo principal es utilizar una nueva estructura de arreglo para cada entrada, cuya representación en bits por elemento sea estrictamente menor que la inicial. Por ejemplo, si los elementos originales eran enteros de 32 bits, la técnica debe calcular un nuevo largo en bits (ya sea un largo máximo fijo para todos, o un largo variable) para que cada elemento del nuevo arreglo ocupe exclusivamente esa cantidad reducida de bits.

Deberá mantener la capacidad de acotar el intervalo utilizando la estructura *sample* del Caso 2, pero la búsqueda del elemento final debe realizarse navegando y decodificando directamente sobre su nuevo arreglo comprimido a nivel de bits. Esto servirá para evaluar si el **tiempo de búsqueda sacrificado** es justificado por el **espacio de memoria ahorrado**.

Importante: A cada grupo se le pedirá trabajar con uno de los algoritmos de la seccion 5. Independiente del algoritmo asignado, el flujo de trabajo general es el siguiente:

1. **Investigar el Algoritmo:** Investigar sobre el algoritmo de compresión asignado y entregar una pequeña descripción del mismo.
2. **Análisis Teórico:** Analizar matemáticamente el algoritmo asignado y formular una **estimación teórica del espacio final**. Para determinar esto, requiere encontrar la cantidad de bits utilizada por cda elemento considerando códigos de largo fijo (Worst Case Entropy) y variable. (ej. si requiere largos variables $\|\mu_i\| = \log_2(\frac{1}{p(\mu_i)})$ u otras aproximaciones). Detalle la deducción de estas fórmulas en su informe.
3. **Implementación a Nivel de Bits:** Mapear los *gaps* utilizando el algoritmo sobre una estructura contigua subyacente (ej. un arreglo de `bytes`, `short` de 8/16 bits, o `uint64_t` usado como mapa de bits continuo).
4. **Búsqueda:** Se realiza búsqueda binaria sobre el *Sample* para hallar un rango $[L, R]$ al igual que en el Caso 2.
5. **Decodificación y Medición:** Implementar la lógica para leer, extraer y decodificar los bits necesarios dentro del rango comprimido para encontrar el valor final buscado. Mida los tiempos para compararlos con los Casos 1 y 2.

Para estimar tamaños base o diseñar empaados manuales, considere la siguiente tabla de referencia de tipos de datos en la arquitectura estándar (C/C++):

Tipo de Dato	Tamaño (sizeof)	Observaciones / Rango Típico
char / int8_t	1 byte (8 bits)	Útil para gaps muy pequeños (0 a 255). Ideal como bloque base de arreglos comprimidos.
short / int16_t	2 bytes (16 bits)	Gaps medianos (0 a 65,535).
int / int32_t	4 bytes (32 bits)	Estándar para valores originales.
long long int / int64_t	8 bytes (64 bits)	Máscaras de bits amplias. Soporta operaciones <i>bitwise</i> («, », &,) eficientes.

3. Metodología de Experimentación

Para validar sus estructuras, debe ejecutar varias pruebas de espacio y tiempo. Considere el uso de herramientas como *Valgrind* para evitar fugas de memoria, dado el tamaño de los datos.

- **Generación de Datos:** Arreglos aleatorios ascendentes con Distribución Lineal y Distribución Normal (Gaussiana) iterando sobre distintas desviaciones estándar.
- **Escala de Pruebas:** Las pruebas deben ser en arreglos de gran tamaño, para los experimentos considere tamaños incrementales en potencias de 2 o de 10 (ej. $10^6, 10^7, 10^8$).
- **Entorno de Medición:** Compile su código fuente utilizando banderas de optimización estándar (ej. `g++ -O3`).

4. Hitos de Entrega y Ponderaciones

El proyecto se desarrollará a lo largo del semestre mediante entregas incrementales a través de la plataforma Siveduc. Los equipos serán de máximo 4 integrantes.

4.1. Hito 1: Implementación (35 %) - *Jueves 28 de Mayo*

Entrega del repositorio en github con el código fuente funcional en lenguaje C++, o el que el grupo estime conveniente.

Estructura del Repositorio

- Los lenguajes compilados deben proveer un script de compilación. En caso de usar C++ puede incluir un `Makefile`.
- Debe incluir un archivo `README.md` con instrucciones de: cómo compilar, cómo ejecutar cada modo, qué argumentos recibe el programa, y qué archivos de salida produce.
- El código debe estar **modularizado**: cada algoritmo de compresión debe implementarse en un archivo de cabecera propio (`.hpp`), separado del archivo principal `main.cpp`. En C++ basta con incluirlos mediante `#include ruta/algoritmo.hpp`, sin necesidad de agregar flags adicionales al `Makefile`. Por ejemplo si se usa C++, se espera un repositorio que contenga al menos:
 - `main.cpp`: punto de entrada del programa.
 - Un `.hpp` por cada algoritmo implementado (Casos 1, 2 y 3).
 - `Makefile` funcional.
 - `README.md` con toda la información necesaria para compilar y ejecutar.

- El código debe usar nombres descriptivos, evitar lógica duplicada y aplicar buenas prácticas. Se recomienda separar el programa en pasos (construcción, búsqueda, medición), para facilitar la medición de los tiempos y espacio.

Modos de Ejecución

El programa debe soportar al menos dos modos, y deben estar documentados en el `README.md`:

- **Modo benchmark:** Ejecuta la generación de datos, construcción de estructuras y medición de tiempos y espacio de forma automática para todos los casos. Ejemplo de uso:

```
./main --benchmark
```

- **Modo archivo:** Recibe como argumento la ruta a un archivo `.csv` con números enteros, construye la estructura correspondiente y luego permite al usuario ingresar valores a buscar de forma interactiva y en qué estructura será buscado. El rango de los números aceptados debe estar especificado en el `README` del repositorio, indicando si el programa trabaja con `int` (usando `atoi`) o `uint64_t` (usando `atoll`); puede consultarse la documentación de estas funciones en [atol, atoll and atof functions in C/C++](#). Ejemplo de uso:

```
./main -i ruta/del/archivo.csv
```

Salidas Esperadas

- En modo benchmark, el programa debe generar un archivo `.csv` con las métricas recopiladas (tiempo de construcción, tiempo de búsqueda y espacio utilizado) para cada caso y tamaño de entrada. Estos datos serán la base de los gráficos del Hito 2.
- En modo archivo, el programa debe indicar si el valor buscado fue encontrado y en qué posición, junto con el tiempo de búsqueda, con múltiples entradas solicitadas en ejecución.
- El programa no debe presentar fugas de memoria ni comportamiento indefinido por desbordamiento (*overflow*). Se recomienda usar `Valgrind` para verificarlo.

4.2. Hito 2: Informe (30 %) - Jueves 11 de Junio

Documento formal claro, objetivo y detallado de máximo 8 páginas (formato IEEE o similar).

- **Introducción y Contexto:** Contexto del problema y declaración de la hipótesis de rendimiento.
- **Metodología y Diseño:** Análisis asintótico de costos de tiempo y espacio de sus soluciones. Inclusión de pseudocódigos para explicar la lógica de *Gap-Coding* y el algoritmo de compresión utilizado. Deducción matemática de las fórmulas de compresión utilizadas.
- **Resultados y Experimentación:** Gráficos que muestren los resultados de tiempo vs tamaño (n) y espacio vs tamaño (n) para las distintas distribuciones.
- **Conclusiones:** Debe responder a la hipótesis inicial y resumir los aspectos más relevantes de los resultados obtenidos.

Es importante realizar múltiples pruebas antes de extraer conclusiones. Adicionalmente, se debe incluir una sección de referencias indicando las fuentes utilizadas.

4.3. Hito 3: Video (35 %) - *Jueves 18 de Junio*

Cápsula de video de **8 minutos aprox** donde participen todos los integrantes del grupo explicando el trabajo realizado.

- **Minuto 1:** Introducción y contexto técnico del problema.
- **Minutos 2-4:** Como se desarrollo la solución. Grabación de pantalla mostrando la ejecución y explicando implementación del código.
- **Minutos 5-6:** Explicación de la experimentación y discusión de los hallazgos más relevantes durante la misma.
- **Minutos 7-8:** Conclusiones y detalle: Principales dificultades y ¿Que harían diferente si tienen que volver a implementar?.

Se recomienda utilizar herramientas como OBS Studio o Zoom para la grabación de pantalla.

Es obligatorio que todos los integrantes aparezcan en el video.

5. Algoritmos de Compresión

A cada grupo de trabajo se le asignará uno de los siguientes algoritmos para aplicar sobre los *gaps*. En todos los casos, el foco debe estar en cómo la técnica reduce los bits por elemento y cómo esto afecta la navegación durante la búsqueda final.

Nota de implementación: La manipulación a nivel de bits en C++ requiere el uso intensivo de operaciones *bitwise* (`<<`, `>>`, `&`, `|`), lo cual puede ser complejo de depurar. Como simplificación permitida, los grupos pueden empaquetar cada código en el tipo de dato más pequeño que lo contenga (`uint8_t`, `uint16_t`, etc.) en lugar de implementar un empaquetado bit a bit estricto. Esta decisión debe declararse explícitamente en el informe, y el análisis teórico de bits por elemento debe realizarse igualmente de forma rigurosa.

5.1. Codificación de Huffman

Algoritmo basado en la frecuencia de aparición de los *gaps*. Asigna una representación en bits más corta a las diferencias más comunes y una más larga a las más raras, minimizando el largo promedio del código.

¿Cómo funciona? Se construye un árbol binario de forma ascendente (*bottom-up*): se toman los dos *gaps* menos frecuentes, se unen en un nodo padre con probabilidad combinada, y se repite hasta tener un único árbol. El código de cada *gap* se obtiene leyendo el camino desde la raíz hasta su hoja (0 = izquierda, 1 = derecha). Por construcción, ningún código es prefijo de otro, lo que permite decodificar sin ambigüedad.

Por ejemplo, dado el siguiente conjunto de *gaps* y sus frecuencias:

Gap	Frecuencia	Código asignado
1	1000	0
2	500	10
5	80	110
7	5	11101011

Los *gaps* frecuentes consumen muy pocos bits, mientras que los raros consumen más, logrando un largo promedio cercano a la entropía del conjunto.

5.2. Codificación de Shannon-Fano

Similar a Huffman, genera representaciones de bits de largo variable según la probabilidad de los *gaps*, pero construyendo el árbol de forma descendente (*top-down*). Puede producir códigos marginalmente peores que el algoritmo de Huffman.

¿Cómo funciona? Se ordenan los *gaps* de mayor a menor frecuencia. Luego se divide el conjunto en dos grupos cuyas frecuencias acumuladas sean lo más similares posible: al grupo superior se le asigna el bit 0 y al inferior el bit 1. El proceso se repite recursivamente sobre cada subgrupo hasta que cada *gap* tiene su propio código único.

Gap	Frecuencia	Código asignado
1	1000	00
2	500	01
5	250	10
7	125	110
9	125	111

5.3. Codificación Elias-Fano

Técnica cuasi-sucinta que, a diferencia de los demás algoritmos de este proyecto, opera directamente sobre los valores del arreglo ordenado en lugar de sobre los *gaps*. Por esta razón, el flujo de trabajo para este grupo difiere levemente: se calcula igualmente el arreglo *GC* y se mantiene el *sample* del Caso 2 para acotar el intervalo $[L, R]$; sin embargo, la búsqueda final dentro de ese intervalo se realiza reconstruyendo los valores originales acumulados y aplicando Elias-Fano sobre ese subarreglo, en lugar de decodificar los *gaps* directamente.

¿Cómo funciona? Dado un universo de valores entre 0 y U , se elige $k = \lfloor \log_2(n) \rfloor$ bits para la parte baja, donde n es la cantidad de elementos. Para cada valor v :

- Los **k bits bajos** (los menos significativos) se almacenan directamente en un arreglo de largo fijo, de forma contigua.
- Los **bits altos** (el cociente $\lfloor v/2^k \rfloor$) se codifican en unario en un segundo arreglo de bits.

Por ejemplo, con $k = 2$ y los valores $\{3, 6, 9, 13\}$:

Valor	Binario	Bits altos (unario)	Bits bajos
3	0011	00 $\rightarrow$ 10	11
6	0110	01 $\rightarrow$ 010	10
9	1001	10 $\rightarrow$ 0010	01
13	1101	11 $\rightarrow$ 00010	01

La parte de bits altos en unario permite saltar eficientemente a una posición aproximada sin decodificar toda la secuencia. El grupo deberá justificar en su informe por qué esta técnica no opera sobre *gaps* y qué implica eso en términos del trade-off espacio-tiempo respecto a los demás grupos.

5.4. Partitioned Elias-Fano (PEF)

Extensión de Elias-Fano clásico que permite aplicar la técnica sobre arreglos de *gaps*, los cuales no son monótonamente crecientes como requiere EF estándar. La solución es dividir el arreglo en bloques y, dentro de cada bloque, acumular los *gaps* localmente para obtener una secuencia creciente sobre la cual sí se puede aplicar EF. Adicionalmente, el parámetro k se elige de forma independiente por bloque, lo que permite adaptarse mejor a distribuciones no uniformes.

¿Cómo funciona? El arreglo de n *gaps* se divide en bloques de tamaño fijo B . Para cada bloque:

1. Se **acumulan localmente** los *gaps* del bloque, obteniendo una secuencia creciente. Por ejemplo, el bloque $[2, 5, 3, 2, 0]$ se convierte en $[2, 7, 10, 12, 12]$.
2. Se calcula el k óptimo local a partir del máximo acumulado U del bloque: $k = \lfloor \log_2(U/B) \rfloor$.
3. Se aplica Elias-Fano estándar con ese k local: los k bits bajos de cada valor se almacenan de forma contigua, y los bits altos ($\lfloor v/2^k \rfloor$) se codifican en unario.
4. Se guarda un **directorio** con el offset de inicio de cada bloque en el bitvector global, el valor base acumulado hasta ese bloque y el k utilizado.

Por ejemplo, con $B = 5$ y el arreglo de *gaps* $[2, 5, 3, 2, 0, 1, 6, 2, 3, 4]$:

Bloque	Gaps	Acumulado local	U	k	Bits bajos por elemento
0	$[2, 5, 3, 2, 0]$	$[2, 7, 10, 12, 12]$	12	1	1 bit
1	$[1, 6, 2, 3, 4]$	$[1, 7, 9, 12, 16]$	16	1	1 bit

El directorio de bloques almacena la información necesaria para saltar directamente a cualquier bloque durante la búsqueda:

Bloque	Offset en bitvector	Base global	k
0	0	0	1
1	X	12	1

Para recuperar el *gap* en la posición i , se determina el bloque $b = \lfloor i/B \rfloor$ y la posición local $j = i \bmod B$. Usando el directorio se salta al offset del bloque b en el bitvector, se decodifican los bits bajos y altos de la posición j , se reconstruye el valor acumulado local $v = (\text{alto} \ll k) \mid \text{bajo}$, y finalmente el *gap* se obtiene como $v - \text{acum}[j - 1]$ (con $\text{acum}[-1] = 0$).

5.5. PForDelta (*Patched Frame of Reference Delta*)

Se basa en analizar un bloque de *gaps* y calcular un ancho máximo fijo en bits (b) que cubra a la gran mayoría de ellos. Luego, todos los elementos del bloque se empaquetan ocupando estrictamente esos b bits. Los valores que no caben en b bits, denominados *outliers*, deben ser manejados de forma separada para no arruinar la compresión del bloque; el grupo deberá proponer e implementar una estrategia para tratarlos.

¿Cómo funciona? Se divide el arreglo de *gaps* en bloques de tamaño fijo (típicamente 128 elementos). Para cada bloque:

1. Se elige b tal que al menos el 90 % de los *gaps* quepan en b bits.
2. Todos los *gaps* que caben se empaquetan directamente usando exactamente b bits cada uno.
3. Los *outliers* (el 10 % restante) deben ser manejados por la estrategia propuesta por el grupo, de forma que su presencia no degrade la compresión del resto del bloque.

Bloque de gaps	[3, 5, 2, 7, 4, 1, 256, 3, 6, ...]
Ancho elegido	$b = 4$ bits (cubre valores 0–15)
Outlier	gap = 256 en posición 6 → requiere tratamiento especial
Resultado	todos los demás en 4 bits + manejo del <i>outlier</i>

Al tener ancho fijo dentro de cada bloque, es posible calcular el desplazamiento de cualquier elemento con una simple multiplicación, lo que hace que la búsqueda dentro del bloque sea de tiempo constante $O(1)$.

5.6. Codificación de Elias (Gamma y Delta)

Códigos universales para enteros positivos cuya longitud depende únicamente de la magnitud del número, sin requerir conocer las frecuencias de antemano.

¿Cómo funciona? Código Gamma (γ): Para codificar el entero n , se escribe $\lfloor \log_2 n \rfloor$ ceros (parte unaria), seguidos de la representación binaria de n (parte binaria). El largo total es $2\lfloor \log_2 n \rfloor + 1$ bits.

Valor	Código γ	Bits usados
1	1	1
2	010	3
3	011	3
4	00100	5
7	00111	5
8	0001000	7

Código Delta (δ): Mejora el código γ para valores más grandes. En vez de codificar $\lfloor \log_2 n \rfloor$ en unario, lo codifica usando el propio código γ , reduciendo el costo para números de magnitud alta. Para valores pequeños ($n \leq 3$), δ es levemente menos eficiente que γ ; para valores grandes, es notoriamente superior.

Ambos códigos son **universales**: sin construir ningún árbol ni calcular frecuencias, garantizan un largo asintóticamente óptimo para cualquier distribución de *gaps* cuya probabilidad decrezca con el valor.

6. Resumen de Entregables e Hitos

Hito	Descripción Breve del Entregable	Ponderación	Fecha de Entrega
1	Código Fuente: Implementación completa (Casos 1, 2 y 3). Debe incluir archivo Makefile y README.md explicativo.	35 %	Jueves 28 de Mayo
2	Informe: Documento formato IEEE (máx. 8 págs) con análisis asintótico, hipótesis, gráficos de tiempos y conclusiones.	30 %	Jueves 11 de Junio
3	Video: Cápsula de 8 minutos con participación de todo el equipo (Introducción, Screen-cast del código, Experimentación, Conclusión).	35 %	Jueves 18 de Junio

Cualquier duda o solicitud de revisión de algoritmos de compresión asignados será atendida por los ayudantes del ramo. Se aconseja iniciar el trabajo experimental lo antes posible.